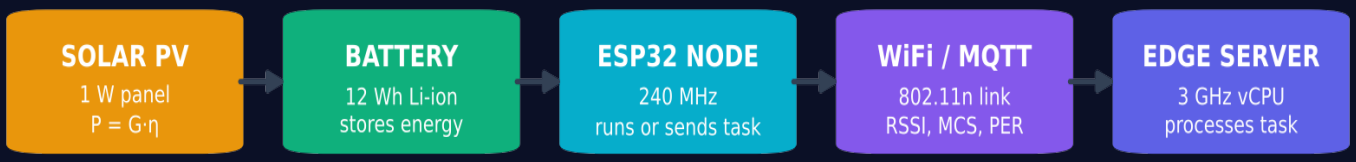
PROJECT GUIDE · COMPLETE EXPLANATION

Renewable-Aware IoT Task Offloading Digital Twin

with an intelligent decision layer: ANN + Fuzzy Logic + ANFIS



Contents

1. The project in one page
2. Physical system & components
3. Software structure (files)
4. Sequential flow: project & every minute
5. Mathematical models in detail
6. The dataset
7. Engineering verification
8. Intelligent layer: ANN, Fuzzy, ANFIS
9. Training, evaluation & results
10. Dashboard walkthrough
11. Web viewer & 3D twin
12. How to run
13. Assumptions & future work
14. Viva questions & answers
15. Glossary

Every number and figure in this guide was generated from the project code and its 7-day dataset.

1 The project in one page

In one sentence

A **digital twin** of a solar-powered IoT sensor node that, every minute, works out whether a new computing task is cheaper to run **on the device (LOCAL)** or to **send over WiFi to an edge server (OFFLOAD)**, using physical models of the sun, battery, processor, wireless link and server. An **ANN + Fuzzy + ANFIS** layer then learns to make the same decision using only the information a real device would have.

What is a digital twin? A software copy of a physical system that behaves like the real thing. You can run it, watch it and experiment with it without building hardware. Here, the "real thing" is a chain of five parts:



Figure 1 — The physical chain mirrored by the twin.

The problem being solved

- A small ESP32 IoT device is powered only by a **1 W solar panel** and a **12 Wh battery**.
- Every minute it must process sensor data (filtering, FFT, anomaly detection, image preprocessing…).
- Running heavy tasks locally is slow and drains the battery. Sending data over WiFi also costs energy, and the cost depends on the signal strength, packet errors and how busy the server is.
- The right choice **changes all the time**, with battery level, weather, distance to the router and server traffic.

What the project delivers

Part	What it does
Physics engine	Simulates 7 days (10,080 minutes) with realistic equations; one task per minute
Exact decision (oracle)	Compares the cost J of LOCAL and OFFLOAD and picks the cheaper. No hard-coded rules
Engineering verification	20 automatic checks that the expected physical relationships appear in the data
Intelligent layer	ANN predicts the offload cost; fuzzy rules (ANFIS) decide; compared with Fuzzy-only and ANN-only
Dashboard (Streamlit)	Live KPIs, 3D twin, 8 engineering plots, ANN/Fuzzy/ANFIS internals, comparison, explanations
Web viewer (HTML)	Offline playback of the dataset: architecture diagram, 3D twin, data flow, module reference

Key numbers

Quantity	Value
Simulated period	7 days × 1,440 minutes = 10,080 tasks
Oracle decisions in the dataset	54.0 % LOCAL · 46.0 % OFFLOAD (emerges from physics)
Battery SOC in the dataset	min 15.2 % · mean 32.0 % · max 57.4 %
ANN prediction accuracy	R^2 = 0.989 (time), 0.987 (energy); median error ≈ 5 %
Best realistic controller	ANN + ANFIS: 96.6 % agreement with the oracle on an unseen week

2 Physical system & components

Each block of the chain is a sub-model with real datasheet-level parameters. All of them are defined at the top of `digital_twin_simulator.py`.

Component	Role	Key parameters
Solar PV panel	Converts sunlight to electrical power	P_{rated} = 1.0 W, η_{PV} = 0.85, daylight 6:00–18:00
Li-ion battery	Stores energy for night and cloudy periods	12.0 Wh, $\eta_{charge} = \eta_{discharge}$ = 0.95, CV taper above 90 %
ESP32 IoT node	Senses, then runs the task locally or sends it	240 MHz, base load 0.14 W, idle CPU 0.1 W, deadline 5 s
WiFi link (MQTT)	Carries task data to the edge and the result back	802.11n MCS0–7, path-loss n = 2.6, shadowing σ = 4.0 dB, P_{tx} = 0.6 W, P_{rx} = 0.33 W
Edge server	Processes offloaded tasks much faster	3 GHz vCPU, utilisation kept below 97 %

The five task types

Each minute brings one IoT monitoring task. Its **type** sets how much work each byte needs (complexity); its **size** is the amount of data collected in the sensing window.

Task type	Complexity (CPU cycles / byte)	Mean size	Typical oracle choice
Sensor Aggregation	60	40 KB	always LOCAL (tiny job)
Signal Filtering	250	80 KB	always LOCAL
FFT Feature Extract	800	160 KB	≈ 50 / 50: depends on battery, link and server
Anomaly Detection	1800	160 KB	mostly OFFLOAD (≈ 85 %)
Image Preprocessing	3000	300 KB	almost always OFFLOAD (≈ 99 %)

Why this is interesting
The middle tasks (FFT, Anomaly Detection) are the "hard cases": the best choice flips with battery level, signal strength and server load. These are where a smart decision maker matters.

3 Software structure (files)

The project is written in Python (NumPy, pandas, Plotly, Streamlit) plus a small HTML/JavaScript viewer. No machine-learning library is used: the ANN, fuzzy system and ANFIS are written from scratch in NumPy.

File	Purpose
digital_twin_simulator.py	**The physics engine.** Parameters, sub-model functions, the stateful twin (one step = one minute), dataset generation
validate_twin.py	20 engineering checks + a "what-if" battery × network experiment
ann_models.py	TinyMLP neural network (NumPy, Adam); the offload-cost **predictor** and the ANN-only classifier
fuzzy_logic.py	Membership functions, linguistic variables and the Mamdani controller with 12 expert rules
anfis.py	ANFIS neuro-fuzzy system: 5 layers, hybrid least-squares + gradient learning, rule explanations
hybrid_offloading.py	Decision-time inputs, the ANN → ANFIS hybrid, training of all models, closed-loop comparison
streamlit_app.py	The live dashboard: controls, KPIs, 3D twin, plots, verification, dataset
dashboard_intelligence.py	Dashboard section for ANN/Fuzzy/ANFIS + the always-visible explanation
twin3d/scene.js, twin3d/index.html	Shared Three.js 3D scene (used by the dashboard and the web viewer)
index.html, app.js, style.css	Offline web viewer that replays the dataset (no physics in JavaScript)
simulation_dataset.csv, web_data.js	Generated dataset (10,080 rows × 38 columns) and the same data for the web viewer

4 Sequential flow

There are two flows to understand: the **project pipeline** (what you run, in which order) and the **per-minute loop** inside the twin.

4.1 Project pipeline

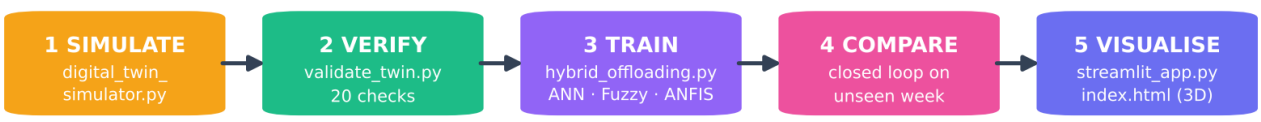


Figure 2 — The five stages of the project.

Stage	What happens	Output
1 Simulate	The twin runs 10,080 one-minute steps with the exact (oracle) decision	simulation_dataset.csv, web_data.js
2 Verify	Spearman correlations, energy balance, decision = argmin(cost), continuity	20 / 20 PASS + what-if table
3 Train	ANN predictor, ANN classifier and ANFIS learn from the first 80 % of the dataset	trained models (in memory)

Stage	What happens	Output
4 Compare	Every controller drives the battery for a NEW week (random seed 7)	agreement, energy, misses, regret
5 Visualise	Dashboard and web viewer show live behaviour, internals and explanations	interactive pages

4.2 What happens in every simulated minute



Figure 3 — The per-minute loop. The battery level at the end of one minute is the starting point of the next, so every decision affects the future (closed loop).

Step	Detail (function in the code)
1 Time	tick += 1; hour of day = (tick mod 1440) / 60
2 Environment	_update_weather (Markov chain at midnight), _update_clouds (Ornstein–Uhlenbeck), _update_position (random walk of distance + correlated shadowing), _update_server_load (daily profile + drift + bursts), ESP32 background load
3 Solar	clear_sky_irradiance(hour) × cloud factor → pv_power()
4 Task	_new_task(): type persists with probability 0.9; size follows an AR(1) process and is larger in working hours
5–7 Options & costs	evaluate_options(): physical model of BOTH options + costs J_local, J_offload (pure function)
8 Decision	controller = None → oracle (cheaper cost). Otherwise controller.decide(inputs) sees only decision-time information
9 Battery	update_soc(): energy of the CHOSEN option + base load against solar → new SOC
10 Record	A dictionary with 38 columns is returned (one dataset row)

5 Mathematical models in detail

Time step Δt = 60 s. Randomness only enters through physical processes (clouds, shadowing, task arrivals, server traffic). **No noise is ever added to costs or to the decision.**

5.1 Solar PV

```
G_clear(t) = 1000 · sin(π (h − 6) / 12) W/m² between 06:00 and 18:00, else 0
G(t) = G_clear(t) · k_cloud(t) k_cloud: OU process around the day mean
P_PV = P_rated · G / G_STC · η_PV (1 W panel, η = 0.85)
```

The daily weather follows a Markov chain (Sunny 0.85, Partly Cloudy 0.60, Overcast 0.30 mean transmittance), so some days charge the battery much more than others.

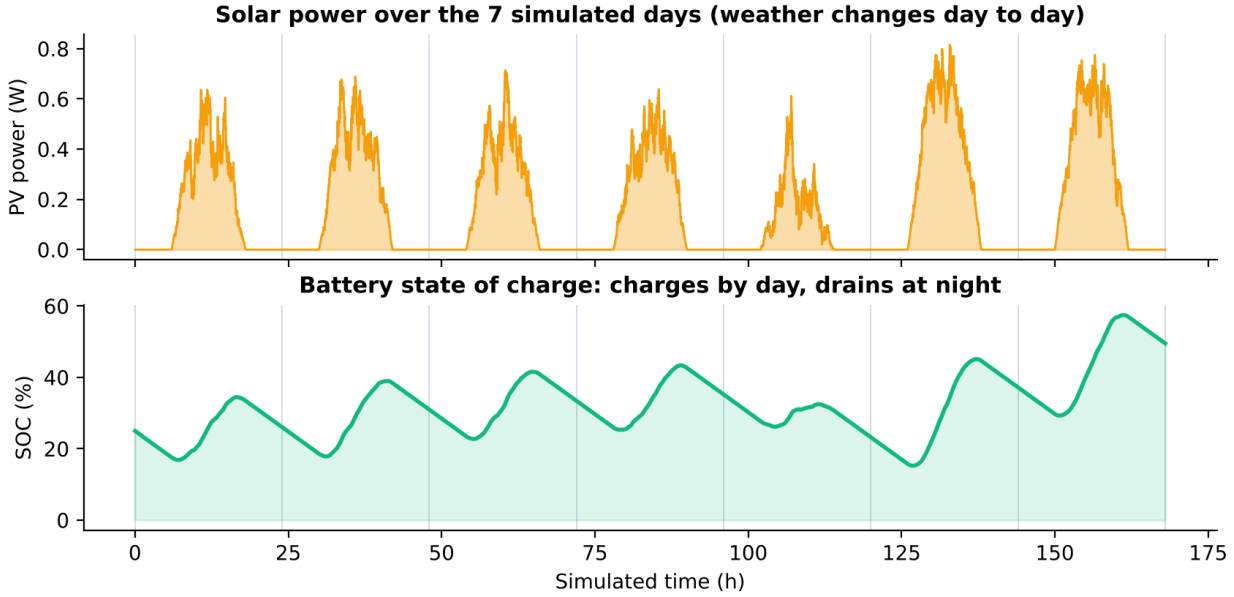


Figure 4 — Solar power and battery SOC over the 7 days of the dataset.

5.2 Battery

```
P_load = P_base + E_task / Δt (P_base = 0.14 W, E_task = energy of the chosen option)
P_net = P_PV − P_load
charging : SOC += η_c · P_charge · Δt / E_bat (charge power × 0.6 above 90 % − CC/CV taper)
discharging: SOC −= P_discharge · Δt / (η_d · E_bat) + tiny self-discharge, SOC kept in
0…100 %
```

5.3 Task

```
C_required = D · c D = task size [bytes], c = complexity [cycles/byte]
D(t) = mean_size · activity(h) · exp(AR(1) state) (continuous, bigger during working hours)
```

5.4 ESP32 local execution

```
CPU_Load = Background + C / (f_ESP32 · T_deadline) · 100 [%]
T_local = C / (f_ESP32 · (1 − Background/100)) [s]
E_local = (P_idle + α · CPU_Load) · T_local [J] (P_idle = 0.10 W, α = 0.0012 W/%)
```

5.5 WiFi link (802.11n + MQTT over TCP/IP)

```
RSSI = RSSI_0 − 10 · n · log10(d / d0) + X_σ (RSSI_0 = −40 dBm, n = 2.6)
MCS = fastest rate whose sensitivity ≤ RSSI (MCS7 at −72 dBm … MCS0 at −91 dBm)
PER = 1 / (1 + 9 · exp(0.7 · margin_dB)) (10 % at the sensitivity limit)
N_tx = 1 / (1 − PER) (max 7 retries)
R_eff = R_MCS · η_MAC · (1 − U_channel) / N_tx
T_tx = (payload + MQTT + TCP/IP headers) · 8 / R_eff
T_network = T_tx + T_result + T_gateway + T_queue
E_offload = P_tx · T_tx + P_rx · (T_edge + T_result + T_gateway + T_queue)
```

Read this as a chain: **further away → weaker RSSI → slower MCS and more packet errors → more retransmissions → longer transmission → more radio energy**. MQTT is modelled through its header overhead and the broker hop.

5.6 Edge server

```
U(t) = daily usage profile + slow drift + decaying traffic bursts
T_edge = C / (f_edge · (1 − U)) M/M/1 form: time explodes as the server saturates
```

5.7 The decision (oracle cost function)

```
λ(SOC) = 1 + 4 · (1 − SOC/100)^1.5 battery-risk price of energy
J_local = λ·E_local/E_ref + T_local/T_dl + 2·[T_local > T_dl]
J_offload = λ·E_offload/E_ref + T_offload/T_dl + 0.3·PER + 0.3·U² + 2·[T_offload > T_dl]
Decision = LOCAL if J_local < J_offload else OFFLOAD
```

Intuition behind the cost

Energy becomes more "expensive" as the battery empties (λ grows from 1 to 5). **Time** is measured against the 5 s deadline, and missing it costs a penalty of 2. Offloading also pays for link unreliability (PER) and server congestion (U^2). Because λ multiplies both energies: at **low SOC with a good link** offloading becomes more attractive, while at **low SOC with a poor link** local execution wins, because the retransmissions cost more than the CPU. Solar affects decisions **only through the battery**.

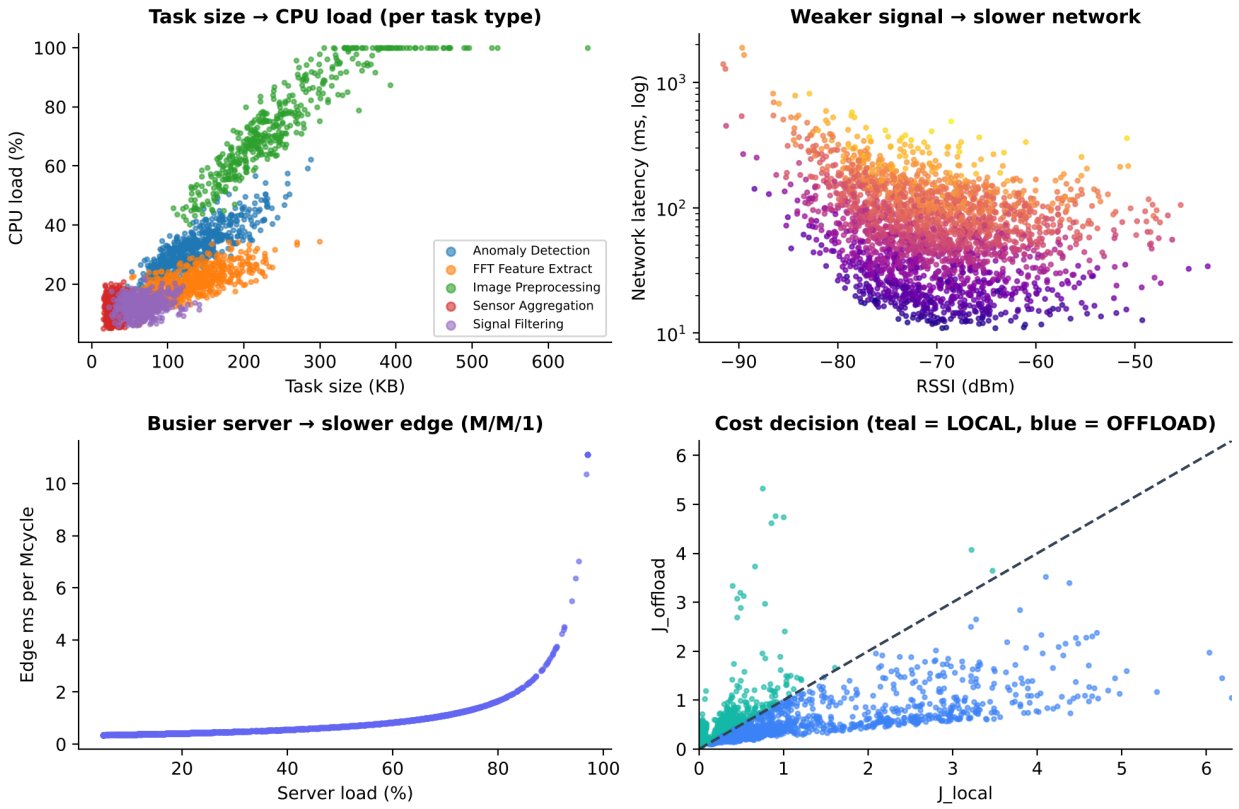


Figure 5 — Relationships that emerge from the equations (2,500 sampled tasks).

6 The dataset (simulation_dataset.csv)

10,080 rows (one per minute) and 38 columns, grouped below.

Group	Columns	Units
Time	Step, Timestamp, Hour, Weather	–
Energy	Solar_Irradiance, Renewable_Power, Renewable_Availability, Load_Power, Battery_Charge_Power, Battery_Discharge_Power, Battery_SOC, dSOC_dt	W/m², W, %, %/min
Task & ESP32	Task_ID, Task_Type, Task_Size, Task_Complexity, CPU_Cycles, Background_Load, CPU_Load, Local_Exec_Time, Local_Energy	KB, cycles/B, Mcycles, %, ms, mJ
Network	Distance, RSSI, WiFi_MCS, Data_Rate, PER, Transmissions, Transmission_Time, Network_Latency, Transmission_Energy, Offload_Energy	m, dBm, Mbps, ms, mJ
Edge	Server_CPU_Load, Edge_Processing_Time, Offload_Total_Time	%, ms
Decision	Battery_Price, Local_Execution_Cost, Offload_Execution_Cost, Decision	λ, J, LOCAL/OFFLOAD

7 Engineering verification (validate_twin.py)

The data is never edited. Instead, 20 checks prove that the expected physics emerges from the equations. Correlations use Spearman ρ (monotonic trend, no linearity assumed). Result: **20 / 20 PASS**.

Relationship checked	Measured
Renewable power ↑ → SOC rate ↑	ρ = +0.94
Daily solar energy ↑ → daily ΔSOC ↑	ρ = +1.00
Task size ↑ → CPU workload ↑ (per type)	min ρ = +0.96
Task complexity ↑ → CPU load ↑	ρ = +0.90
CPU load ↑ → local energy ↑	ρ = +0.94
Task size ↑ → transmission energy ↑	ρ = +0.91
Distance ↑ → RSSI ↓	ρ = −0.78
RSSI ↓ → latency per KB ↑	4.42 > 1.54 > 0.75 > 0.51 ms/KB
RSSI ↓ → retransmissions ↑	ρ = −0.79
Server load ↑ → edge time per Mcycle ↑	ρ = +1.00
SOC ↓ → energy price λ ↑	ρ = −1.00
Energy balance closes	max error 1.2e-05 %
Decision = argmin(cost)	100 %
SOC, RSSI, task size, server load, PV are continuous in time	lag-1 autocorr 0.90–1.00
Battery not stuck at 100 %	0 % of the time

The **what-if experiment** keeps one task fixed (Anomaly Detection, 160 KB, server 40 %) and varies only SOC and RSSI. It shows the battery × network interaction from Section 5.7 in a controlled way.

8 Intelligent layer: ANN + Fuzzy logic + ANFIS

8.1 Why it is needed

The oracle cheats (on purpose)
The exact decision uses the real network latency, radio energy and current server load. A real ESP32 cannot know these **before** it sends the task. It only knows its SOC, the task, the **last** measured RSSI and the **last** load reported by the server. So the exact comparison becomes the **oracle** (the best possible answer), and realistic controllers are built and graded against it.

Decision-time input (known)	Where it comes from
SOC	battery gauge (SOC before this minute's update)
Task_Size, Task_Complexity, Hour	the task itself and the clock
Local_Exec_Time, Local_Energy	the ESP32 can compute its own cost exactly (deterministic CPU)
Last_RSSI	RSSI measured during the previous minute's transmission
Last_Server_Load	load reported by the edge with the previous result

Four controllers are compared: Oracle (perfect knowledge), Fuzzy only, ANN only, and ANN + ANFIS (the hybrid).

8.2 ANN — the predictor (ann_models.py)

An **Artificial Neural Network** learns a function from examples. The **TinyMLP** class is a multilayer perceptron written in NumPy: tanh hidden layers, a linear (regression) or sigmoid (classification) output, mini-batch training with the **Adam** optimiser, and internal standardisation of the inputs.

Network	Inputs	Architecture	Output
OffloadPredictor (used by the hybrid)	log task size, log complexity, last RSSI, last server load, sin/cos(hour)	6 → 24 → 16 → 2	log Offload_Total_Time, log Offload_Energy
AnnOffloadClassifier (ANN-only baseline)	the 6 above + SOC, log local time, log local energy	9 → 24 → 16 → 1	P(OFFLOAD) via sigmoid

- **Training:** forward pass → loss (MSE or cross-entropy) → back-propagation (output error = prediction − target; tanh derivative $1 - a^2$) → Adam update. 60 epochs, batch 128.
- **Why log scale:** offload times range from about 10 ms to several seconds; learning logs means learning ratios.
- **Result (validation):** offload time $R^2 = 0.989$ (median error 5.2 %), offload energy $R^2 = 0.987$ (median error 5.6 %).

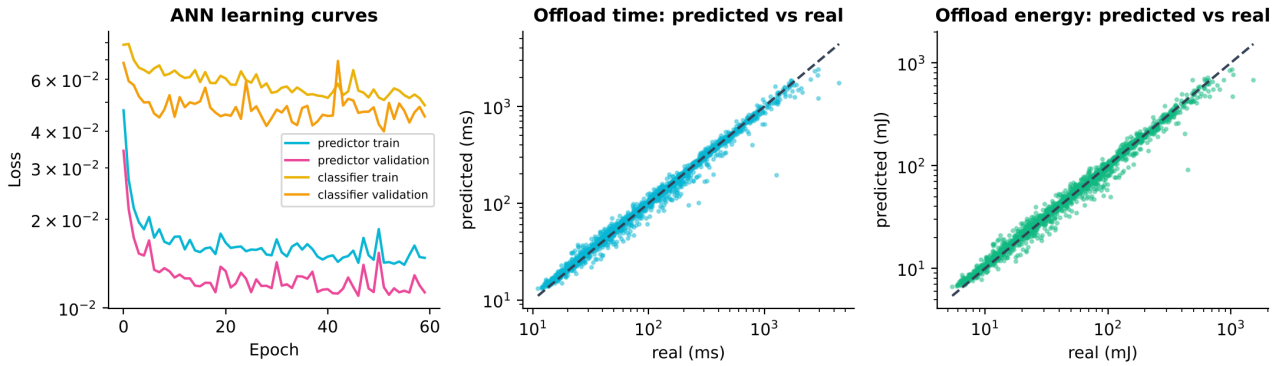


Figure 6 — ANN learning curves and predicted vs real values on validation data.

8.3 Fuzzy logic — the Mamdani controller (fuzzy_logic.py)

Fuzzy logic reasons with vague words instead of hard thresholds. A value belongs to each word with a degree between 0 and 1. This is the **Fuzzy-only** baseline: 4 inputs, 12 hand-written expert rules.

Step	What happens	Implementation
1 Fuzzify	each crisp input → membership degree of each word	trapmf / trimf
2 Rule strength	AND of conditions = minimum of their memberships	np.minimum.reduce
3 Aggregate	each rule clips its output set at its strength; all clipped sets combined by maximum	np.max(np.minimum(...))
4 Defuzzify	centre of gravity of the aggregated shape → offload score 0...1; > 0.5 → OFFLOAD	centroid

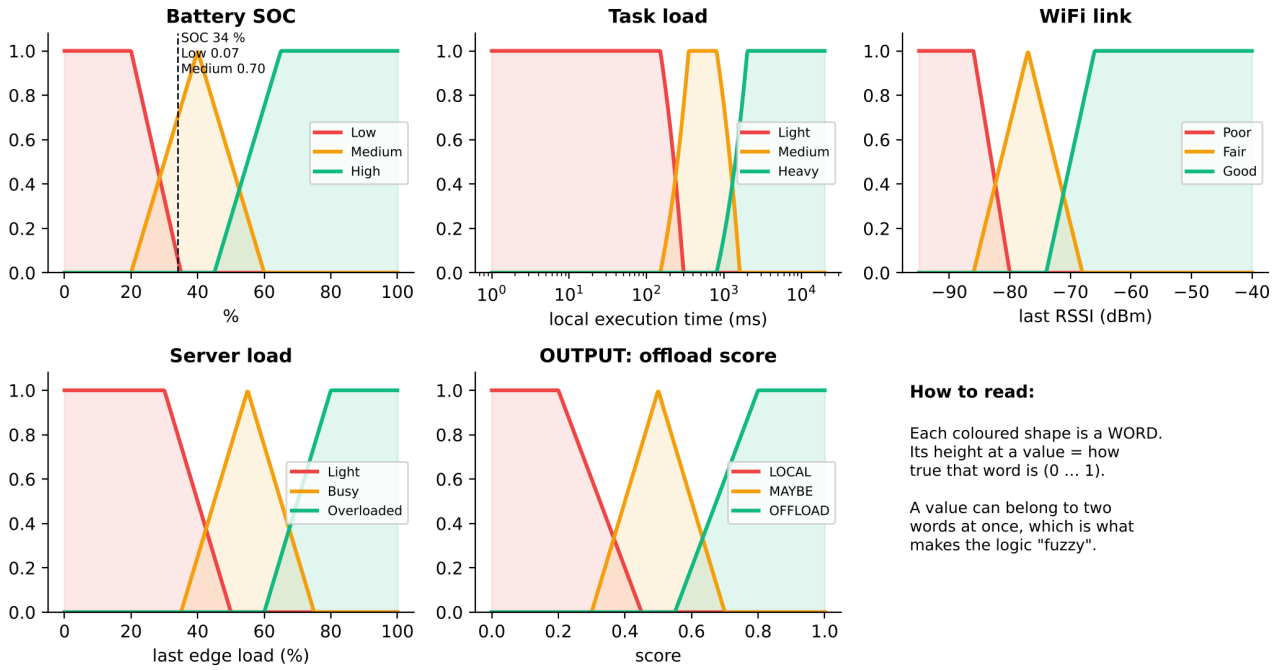


Figure 7 — The fuzzy sets ("words") of the Mamdani controller.

#	Expert rule	Reason
1	IF Task load is Light → LOCAL	radio costs more than the CPU for tiny jobs
2	IF Task load is Medium AND WiFi link is Poor → LOCAL	retransmissions waste energy
3	IF Task load is Medium AND Server load is Overloaded → LOCAL	edge would be slow
4	IF Task load is Medium AND Server load is Busy → LOCAL	waiting for the edge costs RX energy
5	IF Task load is Medium AND Server Light AND WiFi Good → OFFLOAD	good conditions
6	IF Task load is Medium AND Server Light AND WiFi Fair → OFFLOAD	acceptable conditions
7	IF Task load is Medium AND SOC Low AND Server Light → OFFLOAD	save the battery
8	IF Task load is Heavy AND WiFi Good → OFFLOAD	heavy job + good link
9	IF Task load is Heavy AND WiFi Fair → OFFLOAD	
10	IF Task load is Heavy AND WiFi Poor → MAYBE	heavy job but bad link
11	IF Task load is Heavy AND Server Overloaded → MAYBE	heavy job but edge saturated
12	IF Task load is Heavy AND SOC Low → OFFLOAD	local would drain the battery

Worked example

Inputs: SOC = 25 %, local time = 900 ms, last RSSI = −75 dBm, server load = 20 %.
Fuzzify: SOC → Low 0.67, Medium 0.25 · Task → Medium 0.875, Heavy 0.08 · WiFi → Fair 0.78 · Server → Light 1.0
Rules: rule 6 = min(0.875, 1.0, 0.78) = **0.78** → OFFLOAD; rule 7 = min(0.875, 0.67, 1.0) = **0.67** → OFFLOAD; rules 9 and 12 = 0.08.
Defuzzify: centre of gravity = **0.82 → OFFLOAD**. The dashboard shows exactly this list of fired rules.

8.4 ANFIS — fuzzy rules that learn (anfis.py)

ANFIS (Adaptive Neuro-Fuzzy Inference System, Jang 1993) is a fuzzy system built as a network, so its membership shapes and rule outputs can be **trained from data** while staying readable as IF–THEN rules. It uses the first-order Takagi–Sugeno form.

Layer	Operation	Learned?
1 Fuzzification	$\mu(x) = \exp(-(x - c)^2 / 2\sigma^2)$ for every word of every input	yes: c, σ by gradient descent
2 Rule strength	w_r = product of the memberships in rule r (AND)	—
3 Normalisation	$\bar{w}_r = w_r / \Sigma w$	—
4 Rule output	$f_r = p_r \cdot x + q_r$ (a small linear model per rule)	yes: p, q by least squares
5 Output	offload score $y = \Sigma \bar{w}_r \cdot f_r$	—

ANFIS input	Words (low → high)	Source
Battery SOC [%]	Low · Medium · High	battery gauge
Energy ratio = log10(E_offload_pred / E_local)	Offload cheaper · Similar · Local cheaper	**ANN prediction**
Time gain = (T_local − T_offload_pred) / deadline [%]	Small · Large	**ANN prediction**
Server load [%]	Light · Busy	last reported load

Grid partition gives 3 · 3 · 2 · 2 = **36 rules** and 200 trainable parameters.

- **Hybrid learning (each of 30 epochs):** (1) forward pass — membership shapes fixed, all p, q solved at once by ridge least squares; (2) backward pass — p, q fixed, c and σ updated by gradient descent on the mean squared error.
- **Teacher signal:** a soft preference 0.5 + 0.5 · tanh((J_local − J_offload) / 0.25): 1 = OFFLOAD clearly cheaper, 0 = LOCAL clearly cheaper, 0.5 = tie. Smooth targets train better than hard 0/1 labels.
- **Trained on the ANN's predictions, not the true values**, so the rules learn to cope with the ANN's errors.
- **Clipping:** inputs outside the training range (e.g. SOC above 57 %) are clipped to its edge, so the linear rule outputs never extrapolate wildly. The words are therefore relative to the training range.

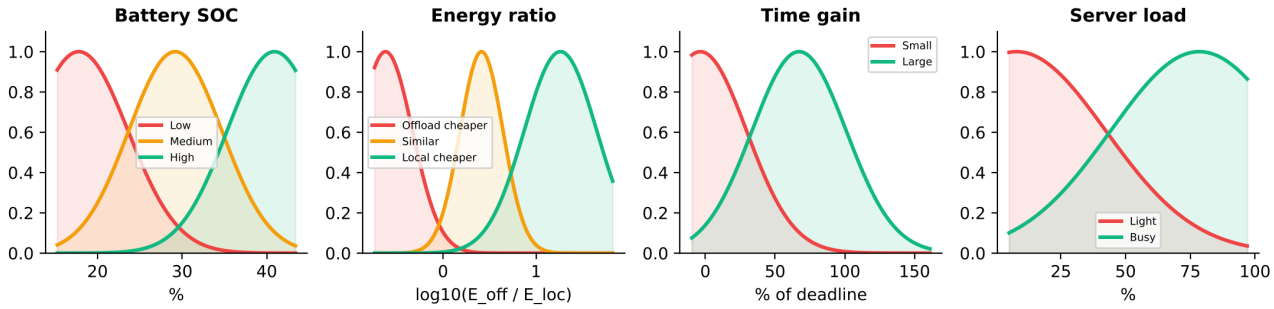


Figure 8 — Membership functions learned by ANFIS.

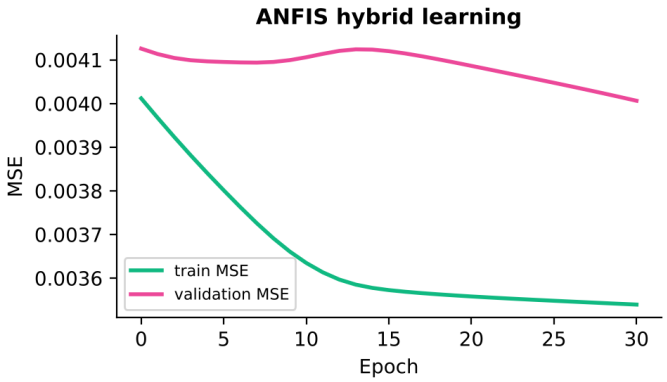


Figure 9 — Hybrid learning lowers both training and validation error.

Most used learned rules (on the validation days; "·" means AND):

Rule	IF	THEN	Score	Usage %
R30	SOC=High · Energy=Similar · Time gain=Small · Server=Busy	LOCAL	0.08	14.2
R34	SOC=High · Energy=Local cheaper · Time gain=Small · Server=Busy	LOCAL	0.00	10.1
R29	SOC=High · Energy=Similar · Time gain=Small · Server=Light	LOCAL	0.00	9.9
R33	SOC=High · Energy=Local cheaper · Time gain=Small · Server=Light	LOCAL	0.00	9.3
R25	SOC=High · Energy=Offload cheaper · Time gain=Small · Server=Light	OFFLOAD	0.77	7.4
R27	SOC=High · Energy=Offload cheaper · Time gain=Large · Server=Light	OFFLOAD	1.00	6.5
R28	SOC=High · Energy=Offload cheaper · Time gain=Large · Server=Busy	OFFLOAD	0.53	5.0
R17	SOC=Medium · Energy=Similar · Time gain=Small · Server=Light	LOCAL	0.09	4.5

8.5 The hybrid: ANN → ANFIS (hybrid_offloading.py)



Figure 10 — Predict → Reason → Decide.

It takes from…	What it gains
the **ANN**	accuracy: a good estimate of what offloading will really cost under uncertain WiFi and server conditions
fuzzy logic	explainability: every decision lists the IF–THEN rules that fired and their weights
ANFIS learning	rules and thresholds tuned from data instead of guessed by an expert

9 Training, evaluation & results

- **Training data:** first 80 % of the 7-day dataset (≈ 5.6 days). **Validation:** last 20 % (≈ 1.4 days, never seen in training).
- **Closed-loop test:** a brand-new week (random seed 7). Each controller drives the battery itself, so a bad decision changes the energy available for every later decision.
- **Metrics:** agreement with oracle (%), task energy (J), deadline misses (> 5 s), mean regret (extra cost J vs the best option), decision time (µs).

Controller	Offline validation accuracy
Fuzzy only	94.84 %
ANN only	98.12 %
ANN + ANFIS	98.07 %

Closed loop on the unseen week:

Controller	Agreement %	Task energy J	Mean time ms	Deadline misses	Regret	Decision µs
Oracle	100.00	911	319	0	0.000	–
Fuzzy only	92.70	978	332	16	0.030	250
ANN only	95.94	951	315	11	0.010	65
ANN + ANFIS	96.56	932	330	9	0.010	170

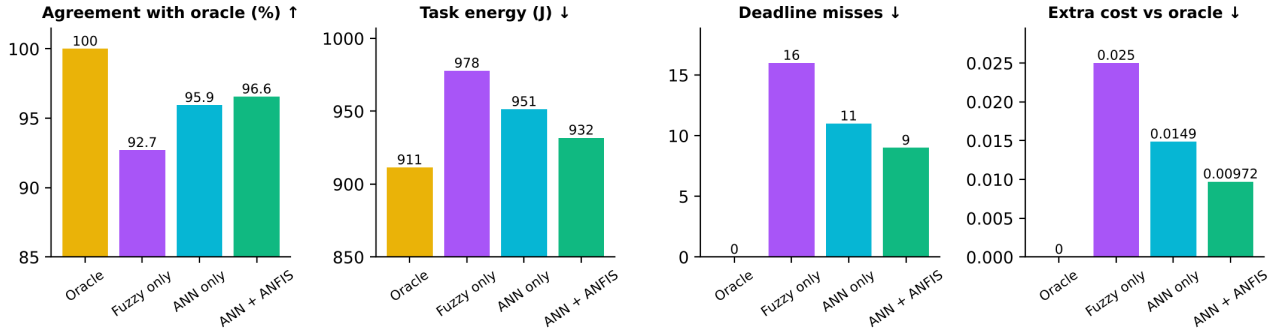


Figure 11 — Closed-loop comparison on the unseen week.

How to interpret the results

Fuzzy only is fully explainable but its hand-chosen thresholds are rough, so it makes the most mistakes. **ANN only** is accurate but a black box. **ANN + ANFIS** is the closest to the oracle on the unseen week: highest agreement, least energy, fewest deadline misses and lowest regret, and it can explain every decision. Offline on the validation days, ANN-only and the hybrid are tied (≈ 98.1 %); the hybrid generalises better to new conditions. The SOC curves of all controllers almost overlap, because task energy is small compared with the base load and the solar input.

10 Dashboard walkthrough

Start it with `streamlit run streamlit_app.py`. The page is laid out top to bottom in this order:

Section	What you see	How to use it
Header + controls	Title, physical chain, Start / Pause / Step / Reset, refresh interval, minutes per refresh	Start runs the twin live; "minutes per refresh" speeds it up (1–60)
Live KPIs	Solar power, SOC (charging/discharging), task, ESP32 CPU, RSSI, latency, edge load, LOCAL/OFFLOAD badge	Current state of every subsystem
3D digital twin	Three.js scene: sun & sky, panel, battery fill, ESP32 glow, WiFi waves, server LEDs; each task animates	Rotate/zoom with the mouse; a packet flies on OFFLOAD, the chip pulses on LOCAL
8 relationship plots	Solar vs time, SOC vs time, size vs CPU load, RSSI vs latency, transmission energy, edge time, J_local vs J_offload, decisions over 24 h	Proof that the equations behave physically
Intelligent layer	Pipeline banner; 4 controller cards (Oracle, Fuzzy, ANN, ANN + ANFIS) with scores; ANN prediction vs reality; ANFIS inputs; fired ANFIS and Mamdani rules; live agreement	Watch how each controller judges the current task
· ANN tab	R^2 and errors, learning curves, predicted vs real scatter	Quality of the predictor
· Fuzzy tab	Membership functions with a marker at the current value, output set, 12 rules	See fuzzification live
· ANFIS tab	Learned sets, hybrid-learning curve, most used rules with scores and usage	What was learned
· Comparison tab	Offline accuracy, closed-loop table, 4 bar charts, SOC traces	The final evaluation
Engineering verification	20 checks and the what-if heatmap (collapsed)	Open to show validity
Dataset	Generate the 7-day dataset (retrains the models), download CSVs, latest records	Reproduce everything
Explanation (always visible)	Six cards: why, ANN, fuzzy, ANFIS, hybrid, how to read the comparison	Teaching / viva aid

The models train automatically on first load (about 10–15 s), with results cached until the dataset changes.

11 Web viewer & 3D twin

`index.html` replays `web_data.js` (the Python output). There is no physics in JavaScript, so the browser shows exactly what the engine computed. Three.js is bundled, so it works offline.

View	Content
Architecture	Animated SVG of the chain with live module values, KPI rings and the decision path of each task
3D Twin	The same Three.js scene as the dashboard (twin3d/scene.js), driven by each dataset row
Data Flow	How a task's data moves through the system step by step
Modules	Reference cards for every sub-model and its equations

3D mapping: irradiance/PV → sun height, sky colour, panel glow, PV→battery flow · SOC → battery fill and colour · CPU load → chip glow · RSSI → WiFi wave colour · server load → server

glow · decision + timings → task animation (timings compressed logarithmically so every stage stays visible). Optional realistic GLB models can be listed in `twin3d/assets/models.js`.

12 How to run

```
pip install -r requirements.txt # numpy, pandas, plotly, streamlit
python digital_twin_simulator.py # 7-day dataset → simulation_dataset.csv + web_data.js
python validate_twin.py # 20 engineering checks + what-if table
python hybrid_offloading.py # train ANN, Fuzzy, ANFIS and print the comparison
streamlit run streamlit_app.py # full dashboard
# or open index.html in a browser (offline playback)
```

13 Assumptions, limitations & future work

Assumptions / limitations	Possible future work
One task per minute; one ESP32 node	Multiple nodes sharing the channel and the edge server
Datasheet-level parameters, not measurements	Calibrate with a real ESP32, INA219 power sensor and real MQTT broker
WiFi/MQTT/TCP modelled analytically, not packet by packet	Packet-level simulation (ns-3) or a hardware-in-the-loop test
Binary decision (all LOCAL or all OFFLOAD)	Partial offloading using the fuzzy score as the offloaded fraction
Models trained offline on SOC 15–57 %	Online learning as new data arrives; more diverse training weeks
No solar forecast in the decision	Add an ANN solar forecaster as an extra ANFIS input

14 Viva questions & answers

Q. **What is a digital twin here?**

A. A software model of the solar → battery → ESP32 → WiFi → edge chain that evolves minute by minute like the real system, so decisions can be studied without hardware.

Q. **Why is the decision not a fixed rule like "offload when solar is high"?**

A. Because the best choice depends on several interacting physical quantities. The twin computes the energy and time of both options and the cheaper one wins, so the behaviour emerges.

Q. **Why does a low battery sometimes favour LOCAL?**

A. λ(SOC) multiplies both energies. With a poor link, retransmissions make the radio cost more energy than the CPU, so local execution becomes cheaper.

Q. **What is the oracle?**

A. The exact cost comparison with perfect knowledge of latency, radio energy and server load. It is the best possible answer and is used to grade realistic controllers.

Q. **Why do you need an ANN?**

A. Before sending, the node cannot know the offload time and energy. The ANN predicts them from known inputs ($R^2 \approx 0.99$).

Q. **Why fuzzy logic instead of only the ANN?**

A. An ANN alone is a black box. Fuzzy rules make every decision explainable in words and handle vague, overlapping conditions smoothly.

Q. **What is the difference between Mamdani and ANFIS?**

A. Mamdani uses hand-written rules with fixed sets and centroid defuzzification. ANFIS is a Sugeno system whose membership shapes and rule outputs are learned from data.

Q. **What is hybrid learning in ANFIS?**

A. Each epoch: least squares solves the rule-output parameters exactly, then gradient descent adjusts the centres and widths of the Gaussian membership functions.

Q. **Why train ANFIS on ANN predictions instead of true values?**

A. Because at run time it only receives predictions; training on them teaches the rules to tolerate the ANN's errors.

Q. **How do you know the models generalise?**

A. Chronological 80/20 split for validation, and a closed-loop test on a different random week where each controller drives the battery itself.

Q. **How do you know the simulation is physically right?**

A. validate_twin.py checks 20 relationships (correlations, energy balance, argmin decision, continuity): all pass.

Q. **Could this run on a real ESP32?**

A. Yes: a 6-24-16-2 MLP and a 36-rule ANFIS need only a few KB of memory and microseconds to evaluate.

15 Glossary

Term	Meaning
SOC	State of charge: battery energy left, in %
PV	Photovoltaic (solar panel)
Irradiance G	Sunlight power per square metre (W/m²)
RSSI	Received signal strength (dBm); closer to 0 = stronger
MCS	Modulation and coding scheme: the WiFi data rate level (MCS0 slowest … MCS7 fastest)
PER	Packet error rate: share of packets that must be re-sent
MQTT	Lightweight publish/subscribe IoT messaging protocol
Edge server	A computer near the devices that processes offloaded tasks
Offloading	Sending a task to another computer to process
J (cost)	Combined score of energy (priced by λ), time and risks; lower is better
λ (lambda)	Battery-risk price of energy, 1 (full) to 5 (empty)
ANN / MLP	Artificial neural network / multilayer perceptron
Adam	Gradient-descent optimiser with adaptive step sizes
Membership function	Curve giving how true a word is for a value (0…1)
Defuzzification	Turning a fuzzy result into one number (here: centre of gravity)
ANFIS	Adaptive Neuro-Fuzzy Inference System: a trainable fuzzy rule base

Term	Meaning
Takagi–Sugeno	Fuzzy model whose rule outputs are linear functions
R^2	Share of variance explained by a model (1 = perfect)
Spearman ρ	Rank correlation measuring a monotonic trend (−1…+1)
Closed loop	Each decision changes the battery, which affects later decisions
Regret	Extra cost compared with the best possible choice